

PROPX and Pareto Optimality for Three Agents and Four Chores: An Exact Certificate

Xiaojian Zeng

PBC School of Finance, Tsinghua University, 100190

xiaojianzeng@foxmail.com

September 7, 2026

Abstract

Every strictly positive rational additive cost instance with three agents, four indivisible chores, and equal shares admits a complete allocation that is proportional up to any chore (PROPX) and Pareto optimal among all complete integral allocations. Empty bundles are permitted. We prove this statement by encoding a counterexample as a formula in twelve real variables and replaying an exact contradiction certificate. The encoding includes all 81 allocations and makes no restriction on cost denominators. A solver-free Python checker verifies 66,972 proof nodes, including 3,131 rational arithmetic lemmas, and derives false with no open assumptions. A second implementation reconstructs the inputs and replays the proof using explicit nonnegative rational combinations for the arithmetic steps. The conclusion concerns the stated finite allocation size; it does not settle compatibility for arbitrary numbers of agents and chores.

1 Statement and context

Let $N = \{0, 1, 2\}$ be the agents and $M = \{0, 1, 2, 3\}$ the chores. For each $i \in N$, an additive cost function is specified by numbers $c_{ie} > 0$, with $c_i(S) = \sum_{e \in S} c_{ie}$ for $S \subseteq M$. We use the normalization $\sum_{e \in M} c_{ie} = 1$. A complete integral allocation $X = (X_0, X_1, X_2)$ is a partition of M ; some parts may be empty. With equal shares, X is *PROPX* if

$$c_i(X_i \setminus \{e\}) \leq \frac{1}{3} \quad (i \in N, e \in X_i). \quad (1)$$

It is *Pareto optimal* (PO) if no complete integral allocation Y satisfies

$$c_i(Y_i) \leq c_i(X_i) \quad \text{for every } i \in N, \quad c_j(Y_j) < c_j(X_j) \quad \text{for some } j \in N. \quad (2)$$

The comparison in (2) includes allocations that are not PROPX.

Theorem 1. *For every matrix $C = (c_{ie}) \in \mathbb{Q}_{>0}^{3 \times 4}$ satisfying $\sum_{e \in M} c_{ie} = 1$ for each $i \in N$, there exists a complete integral allocation satisfying (1) that is Pareto optimal in the sense of (2). Empty bundles are allowed.*

Aziz and Li [1] prove compatibility of weighted PROPX and PO for positive costs when the number of chores is at most the number of agents (Proposition 3.1). Their unequal-share counterexamples for larger instances do not specialize to positive equal-share instances: their parameters require $0 < \varepsilon_i < (1 - ns_i)/2$, which is unavailable when $s_i = 1/n$. Their discussion

after Corollary 3.1 leaves the larger-item, equal-share regime unresolved. Theorem 1 establishes the three-agent, four-chore case immediately beyond the $m \leq n$ boundary.

The author-hosted manuscript of Aziz, Li, Moulin, Wu, and Zhu [2] proves compatibility for two symmetric agents (Proposition 5.1), distinguishes fractional Pareto optimality (Proposition 4.1), and gives examples in which natural algorithms fail. In particular, its three-agent, four-chore leximin example (Example 5.4) is an algorithmic failure, not a nonexistence result. Our conclusion establishes existence throughout this size, without asserting that such an algorithm always finds the allocation. A bounded comparison with these primary sources and targeted searches on September 7, 2026 found no equivalent resolution; this is not a claim of exhaustive priority checking.

Two broader connections help locate the approach. Global Pareto efficiency also appears in the distinct setting of allocation reachability on trees [4]. Certified Boolean encodings and separately checked refutations appear in work on the Erdős–Szekeres problem [3]. These are adjacent subject and methodological references; the deduction below uses no theorem from either work.

2 The counterexample formula

Represent an allocation by a word $a = (a_0, a_1, a_2, a_3) \in \mathcal{A} = N^4$, where a_e is the agent receiving chore e . Thus $|\mathcal{A}| = 81$, including allocations with empty bundles. Define the affine load

$$L_i(a) = \sum_{e: a_e=i} c_{ie}.$$

The following formulas have twelve real variables, the entries of C :

$$P(C) = \bigwedge_{i \in N, e \in M} (c_{ie} > 0) \wedge \bigwedge_{i \in N} \left(\sum_{e \in M} c_{ie} = 1 \right), \quad (3)$$

$$V_a(C) = \bigvee_{e \in M} \left(L_{a_e}(a) - c_{a_e, e} > \frac{1}{3} \right), \quad (4)$$

$$D_{b,a}(C) = \left(\bigwedge_{i \in N} L_i(b) \leq L_i(a) \right) \wedge \left(\bigvee_{i \in N} L_i(b) < L_i(a) \right), \quad (5)$$

$$F(C) = P(C) \wedge \bigwedge_{a \in \mathcal{A}} \left(V_a(C) \vee \bigvee_{\substack{b \in \mathcal{A} \\ b \neq a}} D_{b,a}(C) \right). \quad (6)$$

All cost coefficients and constants are rational. The input consists of 96 assertions: twelve positivity assertions, three row-sum equalities, and 81 allocation-failure assertions. No symmetry reduction, restriction on bundle sizes, or discretization of the cost domain is used.

Lemma 2. *For a strictly positive normalized real matrix C , $F(C)$ holds if and only if every complete integral allocation fails PROPX or is Pareto dominated by another complete integral allocation.*

Proof. For each assigned chore e , the corresponding disjunct in (4) is exactly the negation of its inequality in (1). Every chore has one recipient, so these four disjuncts exhaust the fairness tests. An empty bundle contributes no test. In particular, equality at $1/3$ does not cause a violation. Formula (5) is precisely the weak improvement in all three loads with strict improvement in at least one. The term with $b = a$ is always false, so omitting it changes nothing. Taking the displayed disjunctions and conjunction proves the equivalence. $\square$

It therefore suffices to refute F over the reals. Such a refutation directly excludes every rational counterexample; no approximation of real costs or bound on denominators is needed.

3 Exact verification of the refutation

The electronic supplement contains the formula, the serialized proof, and the checking programs. The proof was produced with Z3 5.1.0 and exported as a topologically ordered directed acyclic graph. Its 118,721 nodes comprise expressions and 66,972 proof inferences. Solver output is used as a proposed certificate: neither the reported unsatisfiability status nor a rule name is accepted without verification.

3.1 Input binding and propositional rules

The standalone checker, `evidence/check_proof.py`, uses only the Python standard library. It reconstructs all 81 allocations and all 96 input assertions from the definitions above, using exact coefficient vectors. The multiset of canonical Boolean formulas must match the certificate’s input multiset. Asserted premises must belong to this verified list. Every node must have the required sort and signature, and every dependency must precede the node.

An affine form is stored as thirteen rational coordinates, twelve for the variables and one for the constant. The checker uses `fractions.Fraction`. Comparisons are interpreted exactly; for example,

$$g \geq 0 \iff -g \leq 0, \quad g < 0 \iff \neg(-g \leq 0), \quad g = 0 \iff (g \leq 0 \wedge -g \leq 0).$$

Weak atoms are normalized only by positive scaling. Boolean constants, associative flattening, idempotence, De Morgan’s laws, and complementary literals give sound simplifications. Residual tautology checks use exhaustive Boolean substitution on the finitely many normalized weak atoms. Treating distinct arithmetic atoms as independent requires a stronger tautology condition and cannot certify a false real identity.

Each proof node carries its set of open local hypotheses. Ordinary inferences inherit the union of the open hypotheses of their premises. The accepted rule classes have the following meanings.

- An assertion is a verified input. A hypothesis introduces and records a local assumption.
- Unit resolution deletes a disjunct only in the presence of a proof of its exact complement. The remainder must be the conclusion; a sole remaining disjunct may itself be compound.
- Modus ponens through Boolean equality, equality transitivity, and congruence check their premise and conclusion structures explicitly. Congruence includes numeral values, so it cannot identify distinct constants merely because they have no children.
- Rewrites and definition axioms must be propositional tautologies after the exact affine interpretation. Negated-disjunction elimination checks the corresponding propositional implication.
- A hypothesis-discharge step starts from false. A hypothesis H can be removed only if the conclusion clause contains $\neg H$. Under the remaining assumptions, falsity of that clause would restore every discharged hypothesis and contradict the premise. No other open assumption disappears.

Unknown rules are rejected. Arithmetic lemmas are checked separately as follows.

3.2 Strict and weak arithmetic inequalities

For an arithmetic inference, the checker conjoins its premises with the negation of its conclusion. In this certificate that conjunction is a list of at most five affine halfspaces, each of the form $g \leq 0$ or $g < 0$. An exact elimination procedure seeks a contradiction.

Suppose two constraints have positive and negative coefficients $a > 0$ and $b < 0$ of a variable x . Multiplying them by $1/a$ and $-1/b$, respectively, and adding eliminates x . Both multipliers are positive. The resulting inequality is strict whenever either summand was strict. Constraints with zero coefficient of x are kept. If coefficients of x have only one sign, dropping the affected constraints weakens the system and cannot create a false contradiction. Positive normalization preserves the inequality direction, and duplicate halfspaces retain the strict version when available.

Every derived inequality is consequently a valid consequence of the starting system. The checker accepts only a constant contradiction: $d \leq 0$ with $d > 0$, or $d < 0$ with $d \geq 0$. It rejects an attempted refutation that leaves an empty feasible system. These checks use exact rational arithmetic, including on equality boundaries.

Lemma 3. *If the described checker accepts a proof graph bound to the inputs of (6), and its root is false with no open hypotheses, then F is unsatisfiable over $\mathbb{R}$.*

Proof. Induct on the topological order of the graph. An asserted conclusion holds under F ; a hypothesis holds under its recorded assumption. Each propositional or equality rule preserves truth under the inherited assumptions, with hypothesis discharge justified as above. Each accepted arithmetic inference has an impossible conjunction of its premises and negated conclusion, by positive combinations of valid inequalities. It therefore also preserves truth. The root would thus be true under any assignment satisfying F and its root assumptions. When the root is false and its assumption set is empty, no such assignment exists. $\square$

3.3 The checked certificate

The saved graph passes these checks. The standalone replay verifies 66,972 proof nodes and 3,131 arithmetic lemmas, deriving false with zero open hypotheses. Table 1 records the principal sizes. Although all 96 input assertions are bound to the encoding, only 69 occur as asserted nodes in this refutation. An inconsistent subset also refutes the full conjunction.

Table 1: Exact certificate and replay counts.

Quantity	Count
Complete labeled allocations	81
Real cost variables	12
Reconstructed input assertions	96
Expression and proof nodes	118,721
Proof nodes	66,972
Arithmetic lemmas	3,131
Maximum initial halfspaces per arithmetic lemma	5
Open hypotheses at the false root	0

A second implementation, `audit/independent_recheck.py`, reads the raw graph without importing either the original checker or the solver. It reconstructs the 96 assertions from bundle sets, sparse rational polynomials, and primitive integer halfspaces, and replays all propositional rules

with its own representation. For the arithmetic steps it uses a different verification route: exact row reduction on subsets of the constraints finds nonnegative rational multipliers. The multipliers are serialized and then replayed by taking their weighted sums. Every variable coefficient must cancel; the resulting constant must be positive, or zero with a positively weighted strict constraint. These weighted sums certify all 3,131 contradictions: 441 positive-constant contradictions and 2,690 strict-zero contradictions. Completeness of the multiplier search is unnecessary, because each returned combination is checked directly. The second replay also ends at false with no open hypotheses.

Proof of Theorem 1. The supplied certificate passes the replay just described, so Lemma 3 refutes (6) over the reals. If a rational matrix in the theorem had no allocation satisfying both requirements, Lemma 2 would make it a real satisfying assignment of F . This contradiction proves the theorem. $\square$

4 Reproduction and scope

The supplement provides the certificate `evidence/proof-dag.json.gz`, its checker, the SMT input and exporter, the separate replay, and its rational combination witnesses in `audit/arithmetic-combinations.json.gz`. The accompanying README gives commands for replay. In an extracted copy, the primary check is

```
python3 -B -S evidence/check_proof.py
```

This command requires no solver or third-party package. Run without `-O`, since assertion checks are part of the verifier; the checker rejects optimized execution. The second replay also uses only the standard library. Regenerating the proposed proof requires Z3 and is unnecessary for verifying the saved certificate. File checksums and a complete file inventory accompany the supplement.

Boundary checks enumerate all allocations and all integral dominators on 25 exact rational instances: uniform costs, exact $1/3$ fairness ties, costs as small as 10^{-12} , distinct nearly concentrated rows, the cited leximin example, and twenty further deterministically generated instances. Each has a joint witness. For uniform costs, all 81 allocations are PO and 54 are PROPX, including 18 fair allocations with an empty bundle. These are finite implementation checks, not the universal proof. Five targeted corruption tests also reject a missing input, an altered threshold, an unsupported arithmetic contradiction, an undischarged false root hypothesis, and a false propositional axiom.

Theorem 1 concerns strictly positive rational normalized costs, equal shares, three agents, and four chores. Zero costs, unequal shares, other allocation sizes, and fractional Pareto optimality are outside its statement. The proof supplies an exact finite refutation for this size; it does not establish a structural rule for larger instances. The general equal-share compatibility question identified in [1] remains beyond the result proved here.

The computational trust base includes the Python runtime, rational arithmetic, JSON and gzip decoding, and the inspected proof interpreters. Neither interpreter has been formally verified. The second implementation reduces reliance on a single representation and arithmetic algorithm, but is not a formal proof-assistant verification.

Computational and AI assistance. AI assistance through OpenAI Codex was used to develop the encoding, certificate exporter and checkers, inspect the mathematical argument, compare primary sources, and prepare the manuscript. A separate model context reviewed the encoding and checker and implemented the second replay and exact boundary checks. This separation does not constitute external peer review or statistically independent evidence. The mathematical evidence is the explicit reduction and reproducible exact certificate, with the trust assumptions stated above.

References

- [1] Haris Aziz and Bo Li. On the incompatibility of weighted PROPX and Pareto optimality for indivisible chores. arXiv:2608.16130v1, 2026. <https://arxiv.org/abs/2608.16130v1>.
- [2] Haris Aziz, Bo Li, Hervé Moulin, Xiaowei Wu, and Xinran Zhu. PROPX fair and efficient allocation of indivisible chores. Author-hosted manuscript, undated. <https://www.cse.unsw.edu.au/~haziz/PROPX%2BP0.pdf>.
- [3] Zijian Zeng. One-Sided Parity Compression for Certified SAT Attacks on the 33-Point Erdős–Szekeres Problem. SSRN working paper, 2026. <https://doi.org/10.2139/ssrn.7334758>.
- [4] Zijian Zeng. Reachable Globally Pareto-Efficient Allocations on Trees Are NP-Complete. *Preprints*, August 2026. <https://doi.org/10.20944/preprints202608.2133.v1>.